

Delete a Listing

We now have the "CRU" of the "CRUD" in place. Now we need to add the delete functionality.

Open the show view at `resources/views/jobs/show.blade.php`. We already have a delete button, but it's not doing anything. It is actually in it's own little form because we can not just use a link. We need to send a **DELETE** request to the server. Update that form to the following:

```
<form
  method="POST"
  action="{{ route('jobs.destroy', $job->id) }}"
  onsubmit="return confirm('Are you sure you want to delete this job?');"
>
  @csrf @method('DELETE')
  <button
    type="submit"
    class="px-4 py-2 bg-red-500 hover:bg-red-600 text-white rounded"
  >
    Delete
  </button>
</form>
```

This form is sending a **DELETE** request to the `jobs.destroy` route. I also added a confirmation. The `@method('DELETE')` directive is a hidden input field that tells Laravel to treat this request as a **DELETE** request. The `@csrf` directive is a hidden input field that Laravel uses to protect against cross-site request forgery (CSRF) attacks.

destroy Method

Now we need to add the `destroy` method to the `JobController`. Open the `JobController` and add the following method:

```
public function destroy(Job $job): RedirectResponse
{
    // If there is a company logo, delete it from storage
    if ($job->company_logo) {
        Storage::delete('public/logos/' . $job->company_logo);
    }

    // Delete the job
    $job->delete();

    return redirect()->route('jobs.index')->with('success', 'Job listing deleted successfully!');
}
```

We are checking for a logo and if there is one, we delete that first. Then we delete the job and redirect with a success message.

Try deleting a job and you should see the logo deleted from storage and the job deleted from the database.

Now we have full CRUD functionality for job listings. Next I want to work on authentication.